

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Интеллектуальный анализ данных»
Тема: «Автоэнкодеры»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;
4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Класс
6	Wisconsin Diagnostic Breast Cancer (WDBC)	2-й признак

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE
import numpy as np
import pandas as pd

class PCA_Manual:
    def __init__(self, n_components):
        self.n_components = n_components
        self.eigen_vectors = None
        self.eigen_values = None
        self.mean = None

    def fit(self, X):
        self.mean = np.mean(X, axis=0)
        X_centered = X - self.mean

        cov_matrix = np.cov(X_centered, rowvar=False)

        eigen_values, eigen_vectors = np.linalg.eig(cov_matrix)
        eigen_values = eigen_values.real
        eigen_vectors = eigen_vectors.real

        sort_index = np.argsort(eigen_values)[::-1]
        eigen_values = eigen_values[sort_index]
        eigen_vectors = eigen_vectors[:, sort_index]

        self.eigen_values = eigen_values[:self.n_components]
        self.eigen_vectors = eigen_vectors[:, :self.n_components]

        return self

    def transform(self, X):
        X_centered = X - self.mean
        return np.dot(X_centered, self.eigen_vectors)

    def fit_transform(self, X):
        self.fit(X)
        return self.transform(X)

data = load_breast_cancer()
X = data.data
y = data.target
target_names = data.target_names

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_tensor = torch.FloatTensor(X_scaled)
input_dim = X.shape[1]

print(f"Размерность данных: {X.shape}")
print(f"Количество классов: {len(np.unique(y))}")
print(f"Классы: {target_names}")
```

```

class Autoencoder(nn.Module):
    def __init__(self, input_dim, bottleneck_dim):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, bottleneck_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(bottleneck_dim, 8),
            nn.ReLU(),
            nn.Linear(8, 16),
            nn.ReLU(),
            nn.Linear(16, input_dim)
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return encoded, decoded

def train_autoencoder(model, data_tensor, epochs=200, lr=0.001):
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    for epoch in range(epochs):
        optimizer.zero_grad()
        _, decoded = model(data_tensor)
        loss = criterion(decoded, data_tensor)
        loss.backward()
        optimizer.step()

    with torch.no_grad():
        encoded, _ = model(data_tensor)
    return encoded.numpy()

print("\n=== Обучение автоэнкодеров ===")
ae_2d_model = Autoencoder(input_dim, bottleneck_dim=2)
ae_3d_model = Autoencoder(input_dim, bottleneck_dim=3)

print("Обучение автоэнкодера (2 компоненты)...")
ae_2d_result = train_autoencoder(ae_2d_model, X_tensor, epochs=200, lr=0.001)

print("Обучение автоэнкодера (3 компоненты)...")
ae_3d_result = train_autoencoder(ae_3d_model, X_tensor, epochs=200, lr=0.001)

print("\n=== Применение PCA (ручная реализация из JIP №1) ===")

pca_full = PCA_Manual(n_components=30)
pca_full.fit(X_scaled)
all_eigen_values = pca_full.eigen_values.copy()
full_info = all_eigen_values.sum()

pca_manual_2d = PCA_Manual(n_components=2)
pca_2d = pca_manual_2d.fit_transform(X_scaled)

pca_manual_3d = PCA_Manual(n_components=3)
pca_3d = pca_manual_3d.fit_transform(X_scaled)

reduce_info_2 = pca_manual_2d.eigen_values.sum()
loss_2 = 100 - (reduce_info_2 / full_info * 100)

reduce_info_3 = pca_manual_3d.eigen_values.sum()
loss_3 = 100 - (reduce_info_3 / full_info * 100)

print(f"Всего собственных значений: {len(all_eigen_values)}")
print(f"Сумма всех собственных значений (полная информация): {full_info:.2f}")
print(f"Потери информативности при переходе к 2 компонентам: {loss_2:.2f}%")
print(f"Потери информативности при переходе к 3 компонентам: {loss_3:.2f}%")

print("\n=== Применение t-SNE с различными значениями perplexity ===")
perplexity_values = [20, 30, 40, 50, 60]
tsne_results_2d = {}
tsne_results_3d = {}

for perp in perplexity_values:
    print(f"t-SNE (2D) с perplexity={perp}...")
    tsne_2d = TSNE(n_components=2, perplexity=perp, init='pca', random_state=42,
                    learning_rate='auto', max_iter=1000)
    tsne_results_2d[perp] = tsne_2d.fit_transform(X_scaled)

```

```

print(f"t-SNE (3D) c perplexity={perp}...")
tsne_3d = TSNE(n_components=3, perplexity=perp, init='pca', random_state=42,
               learning_rate='auto', max_iter=1000)
tsne_results_3d[perp] = tsne_3d.fit_transform(X_scaled)

best_perplexity = 30
tsne_2d_best = tsne_results_2d[best_perplexity]
tsne_3d_best = tsne_results_3d[best_perplexity]

def plot_2d(ax, data, labels, title, show_legend=True):
    scatter = ax.scatter(data[:, 0], data[:, 1], c=labels, cmap='coolwarm',
                        alpha=0.7, edgecolors='k', s=50)
    ax.set_title(title, fontsize=10)
    ax.set_xlabel('Компонента 1')
    ax.set_ylabel('Компонента 2')
    if show_legend:
        legend1 = ax.legend(*scatter.legend_elements(), title="Классы")
        ax.add_artist(legend1)
    ax.grid(True, linestyle='--', alpha=0.3)
    return scatter

def plot_3d(ax, data, labels, title):
    scatter = ax.scatter(data[:, 0], data[:, 1], data[:, 2], c=labels,
                        cmap='coolwarm', alpha=0.7, edgecolors='k', s=50)
    ax.set_title(title, fontsize=10)
    ax.set_xlabel('PC1')
    ax.set_ylabel('PC2')
    ax.set_zlabel('PC3')
    return scatter

fig = plt.figure(figsize=(18, 12))

ax1 = fig.add_subplot(231)
plot_2d(ax1, ae_2d_result, y, 'Autoencoder (2D)', show_legend=False)

ax2 = fig.add_subplot(234, projection='3d')
plot_3d(ax2, ae_3d_result, y, 'Autoencoder (3D)')

ax3 = fig.add_subplot(232)
plot_2d(ax3, pca_2d, y, 'PCA (2D) - JIP №1', show_legend=False)

ax4 = fig.add_subplot(235, projection='3d')
plot_3d(ax4, pca_3d, y, 'PCA (3D) - JIP №1')

ax5 = fig.add_subplot(233)
scatter = plot_2d(ax5, tsne_2d_best, y, f't-SNE (2D, perp={best_perplexity})', show_legend=True)

ax6 = fig.add_subplot(236, projection='3d')
plot_3d(ax6, tsne_3d_best, y, f't-SNE (3D, perp={best_perplexity})')

handles, labels = scatter.legend_elements()
fig.legend(handles, target_names, loc='upper center', ncol=2, fontsize=12, frameon=True)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.subplots_adjust(hspace=0.4)
plt.show()

fig_perp, axes = plt.subplots(1, len(perplexity_values), figsize=(22, 4))

all_tsne_data = np.vstack([tsne_results_2d[perp] for perp in perplexity_values])
x_min, x_max = all_tsne_data[:, 0].min(), all_tsne_data[:, 0].max()
y_min, y_max = all_tsne_data[:, 1].min(), all_tsne_data[:, 1].max()

x_range = x_max - x_min
y_range = y_max - y_min
x_min -= 0.1 * x_range
x_max += 0.1 * x_range
y_min -= 0.1 * y_range
y_max += 0.1 * y_range

for idx, perp in enumerate(perplexity_values):
    axes[idx].scatter(tsne_results_2d[perp][:, 0], tsne_results_2d[perp][:, 1],
                    c=y, cmap='coolwarm', alpha=0.7, s=50)
    axes[idx].set_title(f'perplexity={perp}', fontsize=12)
    axes[idx].grid(True, linestyle='--', alpha=0.3)

    axes[idx].set_xlim(x_min, x_max)
    axes[idx].set_ylim(y_min, y_max)

fig_perp.suptitle('t-SNE с различными значениями perplexity', fontsize=14, y=1.05)
plt.tight_layout()
plt.subplots_adjust(top=0.85)
plt.show()

```

Результат программы:

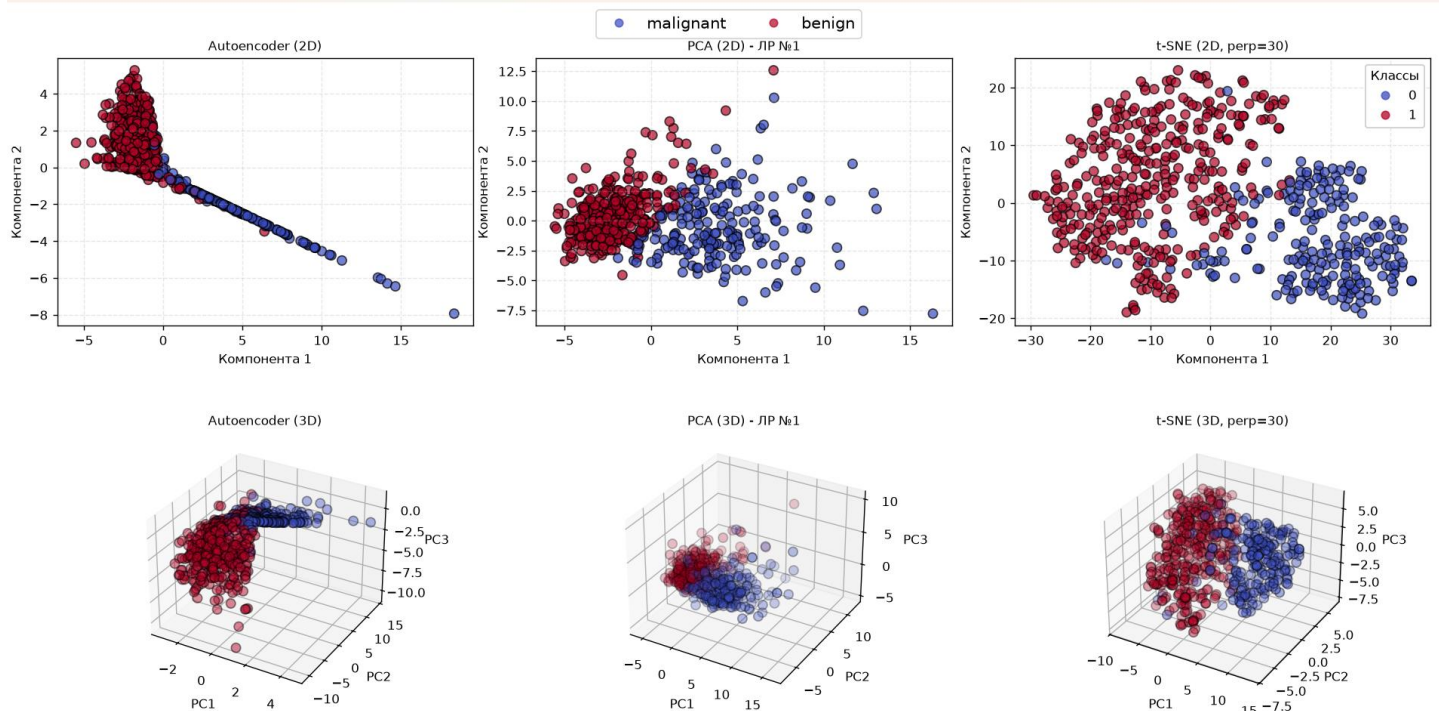
```
C:\Users\Ксения\PycharmProjects\Labs\.venv\Scripts\python.exe C:\Users\Ксения\PycharmProjects\Labs\ИАД_2.py
Размерность данных: (569, 30)
Количество классов: 2
Классы: ['malignant' 'benign']

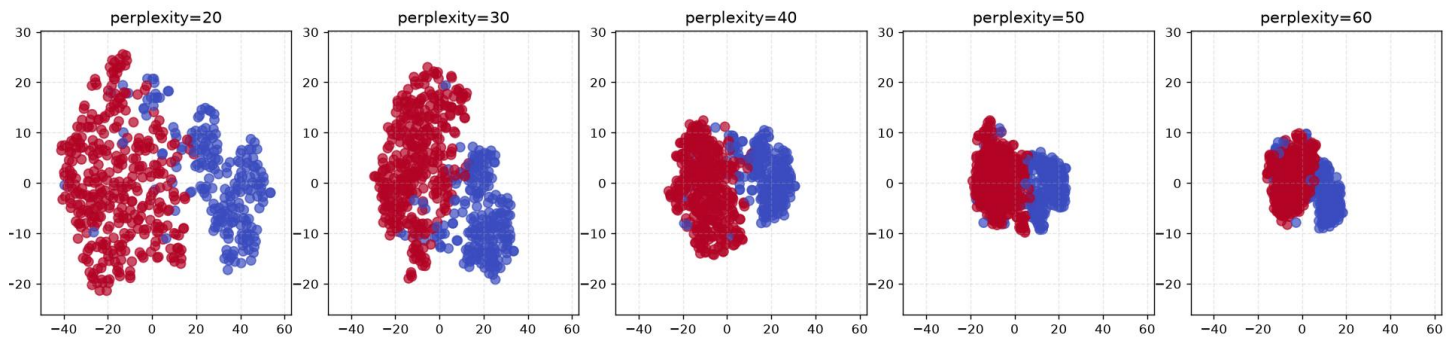
=== Обучение автоэнкодеров ===
Обучение автоэнкодера (2 компоненты)...
Обучение автоэнкодера (3 компоненты)...

=== Применение PCA (ручная реализация из ЛР №1) ===
Всего собственных значений: 30
Сумма всех собственных значений (полная информация): 30.05
Потери информативности при переходе к 2 компонентам: 36.76%
Потери информативности при переходе к 3 компонентам: 27.36%

=== Применение t-SNE с различными значениями perplexity ===
t-SNE (2D) с perplexity=20...
t-SNE (3D) с perplexity=20...
t-SNE (2D) с perplexity=30...
t-SNE (3D) с perplexity=30...
t-SNE (2D) с perplexity=40...
t-SNE (3D) с perplexity=40...
t-SNE (2D) с perplexity=50...
t-SNE (3D) с perplexity=50...
t-SNE (2D) с perplexity=60...
t-SNE (3D) с perplexity=60...

Process finished with exit code 0
```





Выводы о проделанной работе: в ходе лабораторной работы к датасету WDBC (569 объектов, 30 признаков, 2 класса) были применены три метода снижения размерности: автоэнкодер, PCA и t-SNE.

- Автоэнкодер с архитектурой $30 \rightarrow 16 \rightarrow 8 \rightarrow 2/3$, обученный с оптимизатором Adam и функцией потерь MSE, позволил получить двумерное и трёхмерное представления данных с частичным разделением классов malignant и benign.
- Метод PCA (ручная реализация через `numpy.linalg.eig`) показал потери информативности 36.76% при переходе к 2 компонентам и 27.36% — к 3 компонентам, что объясняется высокой исходной размерностью данных (30 признаков). Несмотря на это, визуализация демонстрирует хорошее разделение классов, так как классовая информация сосредоточена в первых главных компонентах.
- Метод t-SNE был протестирован при значениях perplexity от 20 до 60. Оптимальным признано значение perplexity=30, при котором достигается наилучший баланс между чёткостью разделения классов и сохранением структуры данных.

Сравнение методов показало, что t-SNE обеспечивает наилучшее визуальное разделение классов, PCA — быструю визуализацию с сохранением глобальной структуры, а автоэнкодер является гибким обучаемым методом, качество которого зависит от подбора архитектуры и гиперпараметров. Для задач визуализации многомерных медицинских данных наиболее эффективным инструментом является t-SNE с правильно подобранным параметром perplexity.